МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
«БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №2

Специальность ПО-13

Выполнил:
М.А. Романюк студент группы ПО-13
Проверил:
А.А. Крощенко
доц. кафедры ИИТ,
05.03.2026

Брест 2026

Цель работы: закрепить навыки объектно-ориентированного программирования на языке Python

Задание 1:

Вариант 9. Множество вещественных чисел переменной мощности – Предусмотреть возможность объединения двух множеств, вывода на консоль элементов множества, а также метод, определяющий, принадлежит ли указанное значение множеству. Класс должен содержать методы, позволяющие добавлять и удалять элемент в/из множества. Конструктор должен позволять создавать объекты с начальной инициализацией. Реализацию множества осуществить на базе списка. Переопределить метод __eq__, выполняющий сравнение объектов данного типа.

Выполнение:

Код программы

```
class RealSet:
     def __init__(self, elements=None):
self._data = []         if elements:
for element in elements:
self.add(element)

    @property     def
size(self):         return
len(self._data)

    @property
def data(self):
        return self._data.copy()
     def __str__(self):         return "{" + ", ".join([str(round(x, 2)) for
x in self._data]) + "}"
     def __eq__(self, other):         if
not isinstance(other, RealSet):
return False
        return sorted(self._data) == sorted(other.data)
     def contains(self, x):
try:
            return float(x) in self._data
except ValueError:             return
False
     def add(self, x):
try:             num =
float(x)
if num not in
self._data:
self._data.append(num)
else:
print(f"Элемент {num}
```

```
уже существует")
except ValueError:
print("Ошибка: нужно
вещественное число")
     def remove(self, x):         try:
self._data.remove(float(x))         except
ValueError:             print(f"Элемент {x}
не найден")
     def union(self, other_set):         if not
isinstance(other_set, RealSet):
            print("Ошибка: аргумент должен быть объектом RealSet")
return None

        new_data = list(set(self._data + other_set.data))
return RealSet(new_data)


print("=== Множество вещественных чисел (переменной мощности) ===\n")
 set_a = RealSet([1.5, 2.3, 3.7])
set_b = RealSet([2.3, 4.1])
 print(f"A: {set_a}, размер: {set_a.size}")
print(f"B: {set_b}, размер: {set_b.size}")

print("\n=== Тест добавления ===")
set_a.add(5.5) set_a.add(1.5)
set_a.add(7.8) set_a.add(9.2)
print(f"A после добавлений: {set_a}, размер: {set_a.size}")
 print("\n=== Тест contains ===")
print(f"2.3 в A? {set_a.contains(2.3)}")
print(f"10.0 в A? {set_a.contains(10.0)}")
 print("\n=== Тест объединения ===")
set_c = set_a.union(set_b)
print(f"A: {set_a}") print(f"B:
{set_b}")
print(f"Объединение A ∪ B: {set_c}, размер: {set_c.size}")
 print("\n=== Тест удаления ===")
set_a.remove(2.3)
print(f"A после удаления 2.3: {set_a}, размер: {set_a.size}") set_a.remove(15.0)

print("\n=== Тест __eq__ ===") set_d =
RealSet([1.5, 3.7, 5.5, 7.8, 9.2])
print(f"A: {set_a}") print(f"D: {set_d}")
print(f"A == D? {set_a == set_d}")

set_e = RealSet([1.5, 5.5, 3.7, 7.8, 9.2]) print(f"E:
{set_e}")
print(f"A == E? {set_a == set_e}")

print("\n=== Тест с пустым множеством ===") empty_set
= RealSet()
print(f"Пустое множество: {empty_set}, размер: {empty_set.size}") empty_set.add(10.5)
print(f"После добавления: {empty_set}, размер: {empty_set.size}")

print("\n=== Тест с отрицательными числами ===")
set_f = RealSet([-1.5, -2.7, 0, 3.14]) print(f"F:
{set_f}, размер: {set_f.size}")
```

Рисунки с результатами работы программы

```
=== Множество вещественных чисел (переменной мощности) ===

A: {1.5, 2.3, 3.7}, размер: 3
B: {2.3, 4.1}, размер: 2

=== Тест добавления ===
Элемент 1.5 уже существует
A после добавлений: {1.5, 2.3, 3.7, 5.5, 7.8, 9.2}, размер: 6

=== Тест contains ===
2.3 в A? True
10.0 в A? False

=== Тест объединения ===
A: {1.5, 2.3, 3.7, 5.5, 7.8, 9.2}
B: {2.3, 4.1}
Объединение A ∪ B: {1.5, 2.3, 3.7, 4.1, 5.5, 7.8, 9.2}, размер: 7

=== Тест удаления ===
A после удаления 2.3: {1.5, 3.7, 5.5, 7.8, 9.2}, размер: 5
Элемент 15.0 не найден

=== Тест __eq__ ===
A: {1.5, 3.7, 5.5, 7.8, 9.2}
D: {1.5, 3.7, 5.5, 7.8, 9.2}
A == D? True
E: {1.5, 5.5, 3.7, 7.8, 9.2}
A == E? True

=== Тест с пустым множеством ===
Пустое множество: {}, размер: 0
После добавления: {10.5}, размер: 1
```

Задание 2:

Вариант 9. Система Железнодорожная касса. Пассажир делает Заявку на станцию назначения, время и дату поездки. Система регистрирует Заявку и осуществляет поиск подходящего Поезда. Пассажир делает выбор Поезда и получает Счет на Оплату. Администратор вводит номера Поездов, промежуточные и конечные станции, цены.

Выполнение:

Код программы

```
from datetime import datetime
from typing import List, Optional, Dict, Tuple


class TicketStatus:
    PENDING = "Ожидает оплаты"
    PAID = "Оплачен"


class Station:     def
__init__(self, name: str):
self.name = name
     def __str__(self):
return self.name
```

```
class Train:     def __init__(self,
num: str):         self.train_number =
num         self.route: List[Station] =
[]
        self.prices: Dict[Tuple[str, str], float] = {}
     def add_station(self, s: Station) -> None:
self.route.append(s)
     def set_price(self, f: Station, t: Station, p: float) -> None:
self.prices[(f.name, t.name)] = p
     def get_price(self, f: Station, t: Station) -> Optional[float]:
return self.prices.get((f.name, t.name))
     def __str__(self):
        st = " -> ".join([s.name for s in self.route])
return f"Поезд №{self.train_number}: {st}"


class Trip:     def __init__(self, tr: Train, d:
str, tm: str):         self.train = tr
self.date = d         self.time = tm
self.seats = 50
     def __str__(self):
        return f"Рейс {self.train.train_number} | " f"{self.date} {self.time} | Мест:
{self.seats}"


class Request:
    def __init__(self, p: "Passenger", dest: str):
        self.passenger = p
        self.destination = dest
        self.processed = False self.trips:
        List[Trip] = []


class InvData:     def __init__(self,
data: tuple):
        self.passenger = data[0]
self.trip = data[1]
self.amount = data[2]
self.frm = data[3]         self.to
= data[4]


class Invoice:     def __init__(self,
d: InvData):         self.passenger =
d.passenger         self.trip =
d.trip         self.amount = d.amount
self.frm = d.frm         self.to =
d.to
        self.status = TicketStatus.PENDING
self.created = datetime.now()
     def pay(self) -> None:
self.status = TicketStatus.PAID
self.trip.seats -= 1
        print(f"Счет оплачен. Статус: {self.status}")
     def __str__(self):
        return (
```

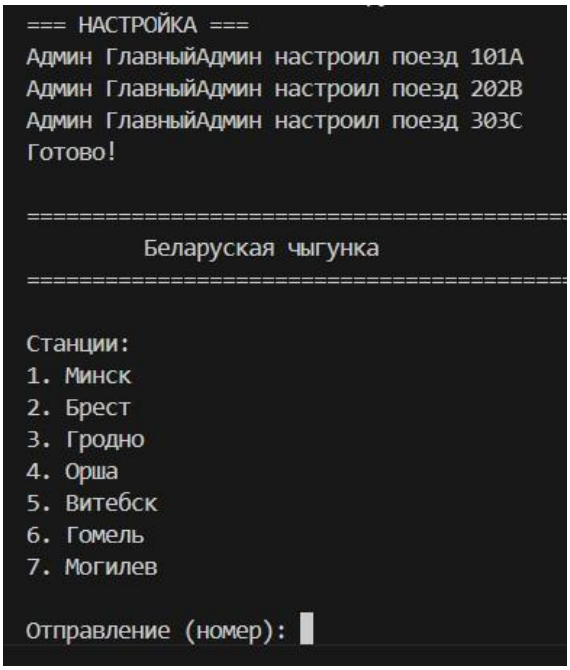
```
            f"Счет: {self.trip.date} {self.trip.time} "
f"{self.frm}->{self.to} = {self.amount} руб. "             f"Статус:
{self.status}"
        )


class Passenger:     def
__init__(self):
        self.reqs: List[Request] = []
     def create_request(self, d: str) -> Request:
        r = Request(self, d)
self.reqs.append(r)         return
r


class Admin:     def __init__(self,
login: str):         self.login =
login
     def add_train(self, sys: "RailwaySystem", num: str) -> Train:
return sys.add_train(num)

      def cfg_train(self, t: Train, st: List[str], pr: Dict[Tuple[int, int], float]) -> None:
        objs = []
        for name in st:
            obj = Station(name)
            t.add_station(obj)
            objs.append(obj)
        for (fi, ti), val in pr.items():
            t.set_price(objs[fi], objs[ti], val)
        print(f"Admин {self.login} настроил поезд {t.train_number}")
     def add_trip(self, sys: "RailwaySystem", t: Train, d: str, tm: str) -> Trip:
return sys.add_trip(t, d, tm)


class RailwaySystem:     def
__init__(self):         self.trains:
List[Train] = []         self.trips:
List[Trip] = []         self.reqs:
List[Request] = []         self.invs:
List[Invoice] = []
     def add_train(self, num: str) -> Train:
t = Train(num)         self.trains.append(t)
return t
     def add_trip(self, t: Train, d: str, tm: str) -> Trip:
tr = Trip(t, d, tm)         self.trips.append(tr)
return tr
     def reg_req(self, r: Request) -> None:
        self.reqs.append(r)
        r.processed = True      def search(self, r: Request, frm:
str, to_: str) -> List[Trip]:         found = []         for tr in
self.trips:             trn = tr.train
            names = [s.name for s in trn.route]
if frm in names and to_ in names:
                fi = names.index(frm)
ti = names.index(to_)                 if fi <
ti and tr.seats > 0:
                    found.append(tr)
```

```
        r.trips = found
return found
     def choose(self, r: Request, tr: Trip, frm: str, to_: str) -> Optional[Invoice]:
trn = tr.train         f_obj = None         t_obj = None         for s in trn.route:
if s.name == frm:
               f_obj = s
    if s.name == to_:
              t_obj = s if f_obj is None or
        t_obj is None:     return None
        price = trn.get_price(f_obj, t_obj) if
        price is None:
            return None
        data_tuple = (r.passenger, tr, price, f_obj, t_obj)
d = InvData(data_tuple)         inv = Invoice(d)
self.invs.append(inv)         return inv


def fmt_row(tr: Trip, f: str, t: str, i: int) -> str:
trn = tr.train
    names = [s.name for s in trn.route]
fi = names.index(f)     ti =
names.index(t)     fo = trn.route[fi]
to = trn.route[ti]     p =
trn.get_price(fo, to)     r = f"{f}-
>{t}"
    return f"{i}. {trn.train_number} {tr.date} " f"{tr.time} {r:15} {p:6} руб. "
f"{tr.seats:3}"


def print_tkt(inv: Invoice, tr: Trip, f: str, t: str) -> None:
print("\n" + "=" * 50)
    print("                 БИЛЕТ КУПЛЕН")
print("=" * 50)
    print(f"Поезд: {tr.train.train_number}")
print(f"Дата: {tr.date}")     print(f"Время:
{tr.time}")     print(f"Маршрут: {f} -> {t}")
print(f"Цена: {inv.amount} руб.")
print(f"Мест: {tr.seats}")     print("=" *
50)


def get_stations(sys: RailwaySystem) -> List[str]:
res = []     for t in sys.trains:         for s in
t.route:
            if s.name not in res:
res.append(s.name)     return res


def show_stations(ss: List[str]) -> None:
print("\nСтанции:")     for i, s in
enumerate(ss):         print(f"{i+1}.
{s}")


def pick_station(ss: List[str], msg: str, ex: int = -1) -> Tuple[int, str]:
    ch = int(input(msg)) - 1 if
    ch < 0 or ch >= len(ss):
```

```
        raise ValueError("Неверный номер")
    if ch == ex:    raise ValueError("Станции должны
    быть разными") return ch, ss[ch]


def buy_proc(sys: RailwaySystem, u: Passenger, f: str, t: str, trips: List[Trip]) -> bool:
print(f"\nРейсов: {len(trips)}")     print("-" * 90)
    print("№  Поезд   Дата       Время   Маршрут          Цена   Места")
print("-" * 90)
     for i, tr in enumerate(trips):
        print(fmt_row(tr, f, t, i + 1))
     ch = int(input("\nВыберите рейс (номер): ")) - 1
if ch < 0 or ch >= len(trips):
print("Неверный номер!")         return False

    tr = trips[ch]     r =
u.create_request(t)
sys.reg_req(r)
    inv = sys.choose(r, tr, f, t)
     if inv:
print(f"\n{inv}")
        pay = input("Оплатить? (1 - да, 0 - нет): ")
if pay == "1":             inv.pay()
            print_tkt(inv, tr, f, t)
else:             print("Отмена")
return True


def setup():
    sys = RailwaySystem()
adm = Admin("ГлавныйАдмин")
print("=== НАСТРОЙКА ===")

    t1 = adm.add_train(sys, "101A")
    adm.cfg_train(t1, ["Минск", "Брест", "Гродно"], {(0, 1): 80, (0, 2): 120, (1, 2): 60})

    t2 = adm.add_train(sys, "202B")
    adm.cfg_train(t2, ["Минск", "Орша", "Витебск"], {(0, 1): 45, (0, 2): 90, (1, 2): 50})

    t3 = adm.add_train(sys, "303C")
    adm.cfg_train(t3, ["Гомель", "Могилев", "Минск"], {(0, 1): 50, (0, 2): 100, (1, 2): 60})

    trips = [
        (t1, "2026-04-15", "08:30"),
        (t1, "2026-04-15", "16:45"),
       (t2, "2026-04-15", "10:15"),
    (t2, "2026-04-16", "09:30"),
       (t3, "2026-04-15", "14:20"), (t3,
        "2026-04-16", "11:40"),    (t1,
        "2026-04-16", "18:30"), ]

    for tr, d, tm in trips:
adm.add_trip(sys, tr, d, tm)
     print("Готово!")
return sys
```

```
def main():
    sys = setup()
user = Passenger()
     while True:         try:
print("\n" + "=" * 70)             print("
Беларуская чыгунка")             print("=" * 70)
             sts = get_stations(sys)
show_stations(sts)
             fc, f = pick_station(sts, "\nОтправление (номер): ")
_, t = pick_station(sts, "Назначение (номер): ", fc)

            r = user.create_request(t)
sys.reg_req(r)
            trips = sys.search(r, f, t)
             if trips:
                buy_proc(sys, user, f, t, trips)
else:                 print(f"\nНет рейсов {f} ->
{t}")
             again = input("\nЕще? (1 - да, 0 - нет): ")
if again != "1":                 print("\nПока!")
break
         except ValueError as e:
print(f"Ошибка! {e}")
except KeyboardInterrupt:
print("\n\nПока!")
break


if __name__ == "__main__":
    main()
```

Рисунки с результатами работы программы



Вывод: закрепил навыки объектно-ориентированного программирования на языке Python.